Field Hours

CSCE 634 Intelligent User Interfaces

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

The adaptation loop

An intelligent interface changes from evidence about this user, not only from a design that was frozen. The loop has 4 beats. You observe an action, infer a small belief, adapt the interface, and take the user's reaction as the next observation. A menu of 12 commands that promotes the 4 most used ones is that loop in miniature. A cutoff between 9 uses and 8 uses is sharp and can be wrong tomorrow.

Adaptation has a cost the static menu does not. The command the user wants may have moved. If they look in the old slot and the command is now 3 places lower, the system has added search time in order to save search time for a more common command. You should move a command when the frequency gap is large, and you should keep a stable way to find all 12, such as a full list that does not reorder.

Four beats, then the user reacts.

1. Observe the command

2. Infer a small belief

3. Reorder the top 4

4. Read the next action

Promote four of twelve

1. The menu has 12 commands. Over the window you counted, four of them dominate.

2. You promote those 4. The command in fifth place, used 8 times against the fourth place's 9, stays in the stable list.

3. The stable list still shows all 12 in a fixed order. A user who needs the rare command once can find it without searching a moving menu.

Problem 1. A menu of 12 promotes the 4 most used commands. The fourth was used 9 times and the fifth 8 times. What is wrong with treating that cutoff as stable truth?

Answer. The gap is 1 use. The loop will reorder on a sharp cutoff that can be wrong tomorrow. Keep a stable list of all 12 so the rare command does not have to be hunted after it moves.

Problem 2. Why can promoting a command add search time?

Answer. The wanted command may now sit 3 places lower than the slot the user remembers. The system spent the user's search time to save search time on a more common command. Move a command only when the frequency gap is large.

Lecture 2 · 90 minutes

Stereotypes and overlays

A user model is what the system stores and then acts on. A stereotype is a bundle of defaults for a group, such as a novice who gets short labels and extra confirms. An overlay records where this person differs. If the stereotype assumes metric units and the user chooses imperial on 4 forms, you store imperial on the overlay and leave the stereotype alone. The working model is the stereotype plus the overlay. On a conflict, the overlay wins.

You should say which observations move someone out of a stereotype. A usable rule is numeric: after 20 finished tasks with fewer than 2 errors, drop the extra confirms. The overlay can be wrong too. Four imperial choices might be one assignment, not a lasting preference. Decay those 4, or let the user edit them, so old observations do not govern a later task. A model you cannot inspect will keep applying a stereotype the person has already left.

What the system believes.

Stereotype: defaults for a group

Overlay: where this person differs

Overlay wins on a conflict

Decay or edit stale observations

Four imperial choices

1. The novice stereotype stores metric units and extra confirms.

2. The user picks imperial on 4 forms. You write imperial on the overlay. Other novices stay metric.

3. On the next form the overlay wins, so the default is imperial. If those 4 picks were one assignment, the user can clear them.

Problem 1. The stereotype says metric, and the overlay says imperial after 4 forms. Which value do you use, and do you change the stereotype?

Answer. You use imperial. On a conflict the overlay wins. The stereotype stays metric for everyone else.

Problem 2. What numeric rule drops the extra novice confirms, and why should 4 imperial picks be allowed to decay?

Answer. After 20 finished tasks with fewer than 2 errors, drop the extra confirms. Four imperial picks may be one assignment, not a preference. If they never decay and cannot be edited, they govern a later task for the wrong reason.

Lecture 3 · 90 minutes

Predicting the next action

A task model is a distribution over the next action given the recent ones. You can estimate it by counts. After File, suppose Save was chosen in 7 of the last 10 times. A predictor that always offers Save is then right 0.7 of

the time in that context. A fancier model must beat 0.7 on actions you did not fit. If it scores 0.65, ship the counts. The mechanism is that conditional frequency.

You can spend a prediction on a highlight, a prefetch, or an automatic action, and those costs differ. Highlighting Save is cheap to ignore. Prefetching a 200 megabyte file because the frequency is 0.7 can be the wrong spend on a slow link. Match the intervention to the cost of a miss. When a wrong action is expensive, do not let a 7 out of 10 count trigger it alone.

Same 0.7, two different spends.

Highlight Save

Cheap to ignore

0.7 can be enough

Prefetch 200 megabytes

Costly if wrong

Wait for a higher count

Seven saves out of ten

1. In the last 10 times the user opened File, Save followed 7 times. The conditional frequency is 0.7.

2. A model that scores 0.65 on held-out actions loses to always offering Save. You ship the counts.

3. You highlight Save. You do not prefetch a 200 megabyte file on that same 0.7, because a miss is expensive.

Problem 1. Save follows File in 7 of the last 10 times. A learned model scores 0.65 on held-out actions in that context. Which do you ship?

Answer. Ship the counts. Always offering Save is right 0.7 of the time, and 0.65 does not beat 0.7. The mechanism is the conditional frequency.

Problem 2. Why may the same 0.7 justify a highlight and not a 200 megabyte prefetch?

Answer. A highlight is cheap to ignore. A prefetch of 200 megabytes on a slow link is not, if the true next file is different. Match the intervention to the cost of a miss. Do not let 7 out of 10 trigger an expensive act alone.

Lecture 4 · 90 minutes

The environment is not a preference

The environment is what is true around the task and is not a lasting preference. Device width, place, and time of day are ordinary features. You can switch to a one-handed layout when the viewport is under 400 pixels and keep a denser layout otherwise. That rule uses 1 measurement. Do not write 'likes mobile' into the user model because someone used a phone once. The width belongs to that situation, not to the preference overlay.

Environment features go stale faster than a stereotype. A location that is 30 minutes old may be a different building. You store the time of the reading and you skip the adaptation when the age passes a limit, such as 2

minutes for a driving signal. Enlarging every control because of a stale driving flag is worse than doing nothing. If the sensor is not allowed, you do not guess the place.

Preference versus situation.

Overlay

Imperial after 4 forms

Decays slowly

A lasting difference

Environment

Width under 400 pixels

Stale after a few minutes

Not an identity

A width is not a taste

1. The viewport is 360 pixels, under the 400 pixel line, so you show the one-handed layout.

2. You do not store 'likes mobile' on the overlay. Tomorrow's desk at 1200 pixels is a new situation.

3. A driving flag older than 2 minutes does not enlarge the controls. If location was never allowed, you do not invent a place.

Problem 1. The viewport is 360 pixels once, at 8 in the morning. Why do you not store 'likes mobile' on the overlay?

Answer. Width is an environment feature, one measurement, not a lasting preference. The one-handed layout applies while the viewport is under 400 pixels. The overlay should not inherit that width as identity.

Problem 2. A driving signal is 30 minutes old, and your limit for that signal is 2 minutes. What do you do?

Answer. You skip the adaptation. A reading that old may be a different situation, and enlarging every control from a stale flag is worse than doing nothing. You stored the time so you can apply the limit.

Lecture 5 · 90 minutes

Act only past a threshold

Mixed initiative means both you and the system can take the next step. The system acts alone only when confidence clears a threshold you set. A highlight may be allowed at 0.7. Deleting a file should wait for something stricter, such as 0.9. If 20 predictions arrive and 5 of them are at or above 0.9, the system acts on those 5 and leaves the other 15 as suggestions. The threshold is a design choice.

Asking has a cost. A dialog on every uncertain step trains people to click through, and the threshold stops mattering. You budget the interruptions: at most 1 ask per 10 actions, and otherwise use a suggestion that does

not block. A false act does the wrong thing. A false ask interrupts a step the user already intended. If the log shows 15 false asks and 1 false act, the asks are too heavy or too frequent for this task.

Confidence decides who acts.

1. Score the next action

2. Highlight at 0.7

3. Act only at 0.9 or above

4. Ask at most 1 time in 10

Five acts out of twenty

1. You set highlight at 0.7 and auto-act at 0.9. Deleting a file uses the stricter line.

2. Of 20 predictions, 5 are at or above 0.9. The system acts on those 5.

3. The other 15 stay suggestions. If the log later shows 15 false asks and 1 false act, you made asking too common.

Problem 1. Of 20 predictions, 5 are at or above 0.9 and the rest are between 0.7 and 0.9. How many automatic acts, and how many highlights, does this rule produce if highlight starts at 0.7?

Answer. The system acts on the 5 predictions at or above 0.9. The other 15 are at least 0.7 and below 0.9, so they are highlights or suggestions, not acts. A delete still uses the 0.9 line.

Problem 2. The log shows 15 false asks and 1 false act, with a budget of 1 ask per 10 actions. What does that log say about the asks?

Answer. The asks are too heavy or too frequent. A dialog trains people to click through, and then the threshold stops mattering. A false ask interrupts a step the user already intended. One false act against 15 false asks is a timid or heavy ask, not a careful act.

Lecture 6 · 90 minutes

Compare adaptive with static

You do not judge an adaptive interface by asking whether it feels smart. Compare it with a static interface on the same tasks. In a small study, 6 people use the adaptive menu and 6 use the stable one. Suppose the adaptive mean is 40 seconds and the static mean is 55. The gap is 15 seconds on this sample. You still have to see the spread, and you have to see the trials where the prediction was wrong.

Count wrong adaptations separately. Suppose the 6 adaptive users make 8 task errors and need 3 recoveries when the guess is wrong. The 6 static users make 14 task errors and 0 such recoveries. Adding recoveries gives 11 events against 14. The adaptive side is still lower, but the 3 recoveries were part of the price. A gain on the first try that disappears by try 5 is a novelty effect. You are not done if you timed try 1.

Count the recovery, not only the clock.

Adaptive, 6 people

Mean time 40 seconds

8 errors plus 3 recoveries

11 costly events

Static, 6 people

Mean time 55 seconds

14 errors, 0 recoveries

14 costly events

Eleven events against fourteen

1. Adaptive users: 8 task errors and 3 recoveries. Sum is 11.

2. Static users: 14 task errors and 0 recoveries. Sum is 14.

3. The adaptive side is lower by 3 events, and only if you counted the recoveries. The 15 second gap, 55 minus 40, is a sample description, not by itself a ship decision.

Problem 1. Adaptive mean time is 40 seconds and static mean time is 55 seconds, with 6 people in each condition. What is the gap, and why is it not yet a decision to ship?

Answer. The gap is 15 seconds. It describes this sample. You still need the spread and the trials where the prediction was wrong. Feeling smart is not the comparison.

Problem 2. Adaptive users make 8 task errors and 3 recoveries. Static users make 14 task errors and 0 recoveries. What are the two event totals, and what does a gain that dies by try 5 mean?

Answer. Adaptive events are 8 + 3 = 11. Static events are 14. The adaptive side is lower only when the 3 recoveries are included. A gain that is gone by try 5 is a novelty effect, so timing only try 1 is not done.

Lecture 7 · 90 minutes

Inspect, undo, and delete the log

The user must be able to see why the interface changed and to undo it. Scrutability means you can open the model. For the menu, the belief to show is that 4 commands were the most frequent over the last 30 days. If you cannot show that sentence, you do not yet understand the adaptation. Undo is one action that restores the previous order. A wrong change with no override is a loss of control.

The model is personal data. Thirty days of commands can reveal a job or a habit, so you set a retention limit before you collect. If the limit is 30 days, the next day drops the oldest day instead of growing an archive. A design that needs a year of logs is a different product from one that needs 7 days, and you should not take the year quietly. Give a control that clears the overlay and returns the stereotype.

A belief the user can read.

1. Show the sentence that fired

2. Name the 30-day window

3. Undo in one action

4. Drop day 1 when day 31 arrives

Why the menu moved

1. The menu promoted 4 commands. The sentence you show is that those 4 were the most frequent over 30 days.

2. One action restores the previous order. If you cannot write the sentence, you do not ship the reorder.

3. On day 31 the oldest day is deleted. A control clears the overlay and the person returns to the stereotype.

Problem 1. What sentence must you be able to show for a menu that promoted 4 commands, and what does undo do?

Answer. That those 4 commands were the most frequent over the last 30 days. Undo is one action that restores the previous order. A wrong change with no override is a loss of control.

Problem 2. The retention limit is 30 days. What happens on the next day, and why is a quiet year of logs a different product from 7 days?

Answer. The oldest day is dropped instead of growing an archive. A year of commands and 7 days of commands are different products. You set the limit before you collect, and you do not take the year quietly.

Lecture 8 · 90 minutes

Specify the loop before you animate

You can now specify an adaptive loop. Name what you observe, whether the belief is a stereotype, an overlay, a next-action count, or an environment feature, and what changes on screen. Set a threshold so a 0.7 guess highlights and a 0.9 guess acts. Compare the result with a static version using time, errors, and recoveries. Say how the user inspects the belief, undoes it, and how many days the log lives.

The task model said Save follows File 7 times out of 10, which is 0.7, and the mixed-initiative rule said you only auto-act at 0.9. You therefore highlight Save and you do not send or delete anything. If the environment sensor is stale by 30 minutes, you also do not let it override that decision. Keep the 0.9 bar even if the overlay says the user is an expert.

The design, not the animation.

What you observe

Which belief you store

The threshold for an act

Inspect, undo, and retention

Highlight, do not send

1. Save follows File in 7 of 10 cases, so the task frequency is 0.7.

2. The act threshold is 0.9. You highlight Save. You do not send the message.

3. A driving or location reading 30 minutes old does not change that. An expert overlay may remove extra confirms and still may not lower the 0.9 bar for a destructive act.

Problem 1. Save follows File 7 times out of 10. Your highlight threshold is 0.7 and your act threshold is 0.9. What does the interface do, and why does an expert overlay not let it send the message?

Answer. It highlights Save and does not send or delete. 0.7 clears the highlight and misses 0.9. The overlay may mark the user as an expert and remove extra confirms, but the lecture keeps the 0.9 bar for a destructive act.

Problem 2. Combine the task count with a stale environment reading. The frequency is 0.7 and the location reading is 30 minutes old, past a 2 minute limit. What may change on screen?

Answer. You may highlight from the 0.7 task count. You do not let the 30 minute location reading override that decision or trigger its own adaptation. Stale environment features are skipped. The act threshold of 0.9 still blocks an automatic send.